SCIENCES SORBONNE UNIVERSITÉ

UFR d'Ingénierie

5 septembre 2025

L2 Informatique (2025-2026)

Logique (UL2IN007)

TP

Thèmes 1 à 5

Thème 1.2.3 - TP

Objectifs

- Logique propositionnelle
  - syntaxe
  - tables de vérité
  - sémantique
  - Interprétations
  - Expressions booléennes
  - Satisfiabilité, validité
  - Conséquence sémantique, équivalence

Exercices

Le but de cette série d'exercices est d'écrire un programme Python qui interprète des formules logiques exprimées sous forme d'une chaîne de caractères.

Les formules sont construites inductivement selon les règles suivantes :
- `TRUE` et `FALSE` sont des formules logiques ;
- si F est une formule atomique, alors F est une formule logique ;
- si F est une formule logique, alors `NOT` F est une formule logique ;
- si F_1 et F_2 sont des formules logiques, alors F_1 `AND` F_2, F_1 `OR` F_2 et F_1 `IMPLIES` F_2 sont des formules logiques.

Les formules atomiques seront représentées par des chaînes de caractères. Les formules sont parenthésées pour éviter toute ambguïté. Voici quelques exemples de formules :
- `"bonjour"`
- `"TRUE IMPLIES FALSE"`
- `"(NOT b) AND (q OR r)"`

Exercice 1

Nous considérons dans un premier temps des formules construites uniquement à partir de `TRUE` et `FALSE`.

Lorsque l'on construit l'arbre de syntaxe abstraite d'une formule, on commence par identifier le connecteur principal, qui sera l'étiquette de la racine de l'arbre. Vous disposez d'une fonction

```
decompose(formule: str)-> (str, list[str])
```

qui prend une formule en paramètre et qui renvoie le couple formé du connecteur principal et de la liste des sous-formules (0, 1 ou 2). Par exemple :
- `decompose("p")` renvoie `('p', [])`
- `decompose("NOT b")` renvoie `('NOT', ['b'])`
- `decompose("(NOT b)AND (q OR r)")` renvoie `('AND', ['(NOT b)','(q OR r)'])`

Lorsque la formule est représentée par un AST,
- soit l'arbre est réduit à une feuille et la valeur de la formule correspond à l'évaluation de la feuille,
- soit la valeur est obtenue en évaluant le(s) sous-arbre(s) et en leur appliquant l'opération représentée à la racine. Les sous-arbres correspondent ici aux éléments de la liste renvoyée par `decompose`.

Les formules traitées dans cet exercice sont construites uniquement à partir de `TRUE` et `FALSE`.

Question 1
Écrivez une fonction récursive `eval_TF (formule : str)-> bool` qui prend en paramètre une formule et qui calcule la valeur booléenne obtenue lors de l'interprétation de cette formule. Par exemple,

- `eval_TF("TRUE AND FALSE")` renvoie `False`,
- `eval_TF("(TRUE AND FALSE)OR (TRUE OR (TRUE AND FALSE))")` renvoie `True`.

Si nécessaire, vous pouvez utiliser la fonction `strip` de la bibliothèque Python pour éliminer d'éventuels espaces inutiles en début ou fin de chaîne.

Exercice 2

Dans le cas général, une formule est construite à partir de formules atomiques. Pour interpréter la formule, il faut donc :

- identifier les formules atomiques qui y figurent,
- interpréter les formules atomiques puis interpréter la formule complète de manière récursive.

Pour identifier les formules atomiques, nous allons réutiliser la fonction

`decompose(formule: str)-> (str, list[str])`

définie dans l'exercice précédent. En effet, les formules atomiques (mais attention, `TRUE` et `FALSE` aussi) étiquettent les feuilles de l'arbre de syntaxe abstraite et celles-ci peuvent être facilement identifiées au moyen de la fonction :
`decompose("p")` renvoie `('p', [])`

Question 1
Écrivez la fonction `genere_fa(formule : str)-> set[str]` qui renvoie l'ensemble des formules atomiques contenues dans formule. Par exemple :

- `genere_fa("TRUE")` renvoie `set()`
- `genere_fa("p AND FALSE")` renvoie `{'p'}`
- `genere_fa("hello OR (aurevoir AND salut)")` renvoie `{'hello', 'aurevoir', 'salut'}`

Pour interpréter une formule, il faut commencer par interpréter les formules atomiques que nous venons d'isoler. Nous supposons que nous disposons d'un dictionnaire qui associe une valeur booléenne à chaque formule atomique.

Par exemple, `{'p': True, 'q': False, 'r': True, 'bonjour': True, 'a': False}` est une valuation qui peut être utilisée pour évaluer les formules `"p AND q"` ou `"r IMPLIES bonjour"`.

Question 2
Écrivez une fonction récursive `eval_formule(formule : str, valuation : dict)-> bool` qui prend en paramètre une formule et une valuation et qui renvoie la valeur booléenne correspondant à l'interprétation de la formule.

Vous disposez toujours de la fonction `decompose(formule: str)-> (str, list[str])` et vous pouvez faire l'hypothèse que la valuation est correcte, c'est-à-dire qu'elle définit bien une valeur pour toutes les formules atomiques de la formule.

Exercice 3

Nous voulons maintenant construire la table de vérité d'une formule. Cela suppose :

- de construire toutes les interprétations possibles de ses formules atomiques,
- d'interpréter la formule pour chacune de ces interprétations.

Pour construire l'ensemble des interprétations, nous allons commencer par construire l'ensemble des parties de l'ensemble des formules atomiques, puis associer une interprétation v_P à chaque partie P selon la règle suivante :

si la formule atomique q appartient à P, alors $v_P(q) =$ `True` sinon $v_P(q) =$ `False`

Par exemple, pour une formule F dont l'ensemble des formules atomiques est $FA_F =$ `{'p', 'q'}`, l'ensemble des parties de FA_F est `{{}, {'p'}, {'q'}, {'p', 'q'}}`. Les valuations correspondantes sont

- `{'p': False, 'q': False}` pour $P =$ `{}`
- `{'p': True, 'q': False}` pour $P =$ `{'p'}`
- `{'p': False, 'q': True}` pour $P =$ `{'q'}`
- `{'p': True, 'q': True}` pour $P =$ `{'p', 'q'}`

L'ensemble $\mathcal{P}(E)$ des parties de E peut être construit en utilisant le mécanisme suivant :

- Si E est l'ensemble vide, alors $\mathcal{P}(E) = \{\emptyset\}$
- sinon E peut être vu comme l'union d'un ensemble E_1 (potentiellement vide) avec un singleton $\{x\}$. $\mathcal{P}(E)$ peut alors être construit à partir de $\mathcal{P}(E_1)$ en appliquant
  $\mathcal{P}(E) = \mathcal{P}(\{x\} \cup E_1) = \mathcal{P}(E_1) \cup \{\ \{x\} \cup A \mid A \in \mathcal{P}(E_1)\ \}$

Question 1

Écrivez une fonction `genere_sous_ensembles(ensemble : set)-> list[set]` qui prend en paramètre un ensemble et qui renvoie la liste composée de l'ensemble de ses parties.

Vous pourrez, si vous l'estimez utile, utiliser la fonction `deepcopy` du module `copy` qui permet de dupliquer un ensemble et ses éléments.

Question 2

Écrivez une fonction `genere_interpretations(fa : set)-> list[dict]` qui prend en paramètre un ensemble de formules atomiques et qui renvoie la liste composée de toutes les interprétations que l'on peut construire pour cet ensemble de formules.

Vous pourrez, si vous l'estimez utile, écrire d'abord une fonction qui construit une interprétation à partir d'un ensemble de formules atomiques et d'une partie de cet ensemble. Par exemple, `{'p': True, 'q': False}` pour $FA_F =$ `{'p', 'q'}` et $P =$ `{'p'}`.

Question 3

Écrivez une fonction `cree_table(formule : str)-> list[list]` qui prend en paramètre une formule et qui construit la table de vérité pour cette formule. La fonction insère dans la liste une première liste contenant les formules atomiques de la formule à évaluer et la formule elle-même. Les autres éléments associent à chaque valuation des formules atomiques l'interprétation correspondante pour la formule.

Par exemple, `cree_table("p AND q")` renvoie :

```
[['p', 'q', 'p AND q'], [False, False, False], [True, False, False], [False, True, False], [True, True, True]]
```

Vous avez accès aux différentes fonctions écrites jusqu'à présent :

- `genere_fa(formule : str)-> set[str]`
- `eval_formule(formule : str, valuation : dict)-> bool`
- `genere_valuations(fa : set)-> list[dict]`

Exercice 4

Pour écrire les fonctions de cet exercice, vous pouvez utiliser les fonctions définies précédemment, à l'exception de celle qui construit la table de vérité puisqu'il est souvent inutile de calculer toutes les interprétations pour déterminer si une formule est satisfiable ou valide.

Question 1
Écrivez la fonction `is_satisfiable(formule : str)-> bool` qui renvoie `True` si la formule en paramètre est satisfiable, `False` sinon.

Question 2
Écrivez la fonction `is_valid(formule : str)-> bool` qui renvoie `True` si la formule en paramètre est valide, `False` sinon.

Question 3
Écrivez la fonction `models(formule : str)-> list[dict]` qui prend en paramètre une formule et qui construit la liste des modèles de cette formule, i.e., des interprétations pour lesquelles la formule s'interprète à `True`.

Par exemple, `models("p OR q")` renvoie :

```
[{'p': False, 'q': True}, {'p': True, 'q': False}, {'p': True, 'q': True}]
```

Une formule F_2 est la conséquence sémantique d'une formule F_1, noté $F_1 \models F_2$, si et seulement si pour toute interprétation I telle que $[F_1]^I = 1$, on a $[F_2]^I = 1$.

Question 4
Écrivez la fonction `consequence_sem(formule1 : str, formule2 : str)-> bool` qui renvoie `True` si `formule2` est une conséquence sémantique de `formule1`, `False` sinon.

Exercice 5

Il est temps maintenant de s'intéresser à la boîte à outils.

La fonction `decompose(formule : str)-> (str, list[str])` renvoie le couple formé du connecteur principal de la formule (la racine de l'arbre de syntaxe abstraite) et de la liste de ses sous-formules. Nous allons l'écrire en deux temps : décomposition d'une formule simple (au plus un connecteur) puis décomposition d'une formule générale.

Pour écrire les fonctions de cet exercice, vous pourrez être amenés à utiliser les fonctions `find`, `split` et `strip` qui opèrent sur les chaînes de caractères Python.

Question 1
Écrivez une première version de la fonction `decompose` que nous appellerons `decompose_simple` et qui s'applique aux formules sans parenthèses qui contiennent donc au plus un connecteur.

Par exemple,

- `decompose_simple("p")` renvoie `('p', [])`,
- `decompose_simple("NOT p")` renvoie `('NOT', ['p'])`,
- `decompose_simple("p IMPLIES q")` renvoie `('IMPLIES', ['p', 'q'])`.

Vous pouvez faire l'hypothèse que la formule passée en paramètre est bien sans parenthèses.

Pour écrire la fonction `decompose` qui décompose une formule dans le cas général, nous allons nous intéresser à l'équilibre des parenthèses dans la formule.

- si le nombre de parenthèses ouvrantes ne correspond pas au nombre de parenthèses fermantes, la formule est mal parenthésée et la fonction renvoie `None` ;
- si la formule commence par une parenthèse ouvrante et se termine par une parenthèse fermante, ces parenthèses sont inutiles, on reprend l'analyse en les supprimant ;
- sinon on recherche la première position `i` à partir de laquelle le nombre de parenthèses ouvrantes et fermantes s'équilibre. Le connecteur principal est le premier qui suit cette position.

Question 2

Écrivez a fonction `decompose(formule : str)-> (str, list[str])` qui renvoie le couple formé du connecteur principal de la formule et de la liste de ses sous-formules. La fonction `decompose` doit traiter tous les cas, y compris les cas simples (vous pouvez faire appel à la fonction `decompose_simple)`. Vous disposez de la fonction

```
trouve_prochain_connec(formule : str, index : int)-> (str, int)
```

qui renvoie le premier connecteur apparaissant dans formule à partir de la position `index` et son indice dans `formule`. Par exemple, `trouve_prochain_connec("toto IMPLIES titi", 2)` renvoie `('IMPLIES', 5)`.

Question 3

Écrivez a fonction

```
trouve_prochain_connec(formule : str, index : int)-> (str, int)
```

qui renvoie le nom et la position dans formule du premier connecteur dont la position est au moins égale à `index`. Par exemple, `trouve_prochain_connec("(NOT b)AND (q OR r)", 2)` renvoie `('AND', 8)`.

Thème 4.5 - TP

Objectifs

- Décuction naturelle

Exercices

Preuves sur Edukera, accès sur moodle.